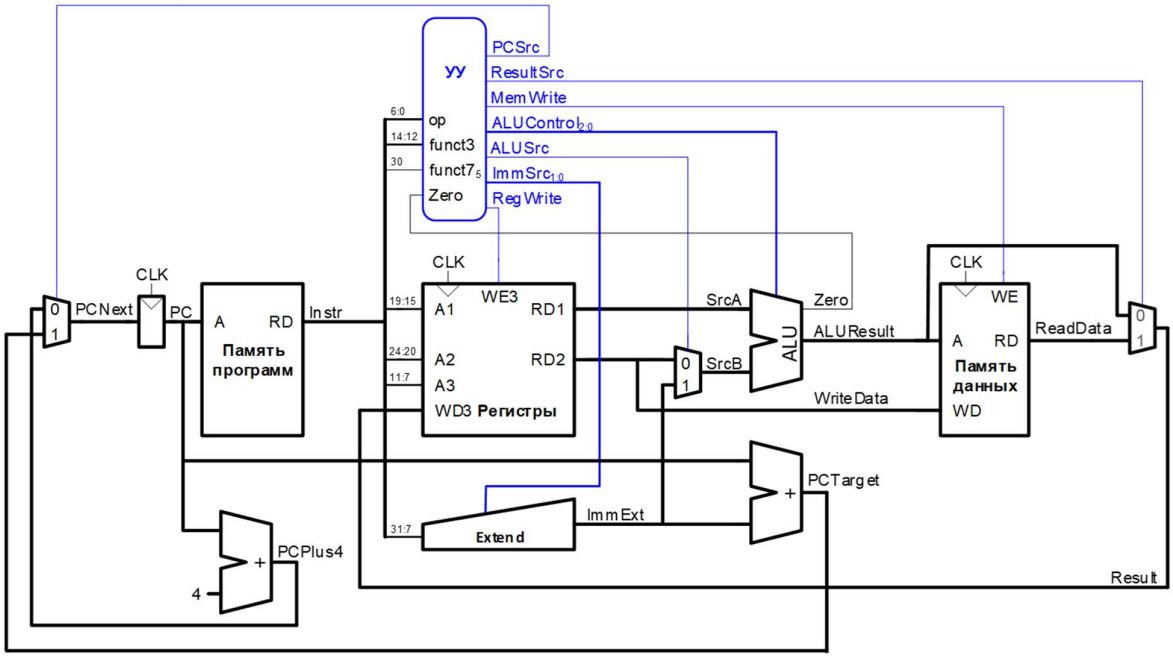
Лабораторная работа 3. Процессор с архитектурой RISC-V

Необходимо выполнить разработку процессора с архитектурой RISC-V. Для зачета лабораторной работы процессор должен уметь выполнять минимум **три** команды **различных** типов.

На рис приведена предлагаемая архитектура процессора. Она может быть изменена/дополнена при необходимости.



Форматы команд:

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Список команд:

RV32I Base Integer Instructions

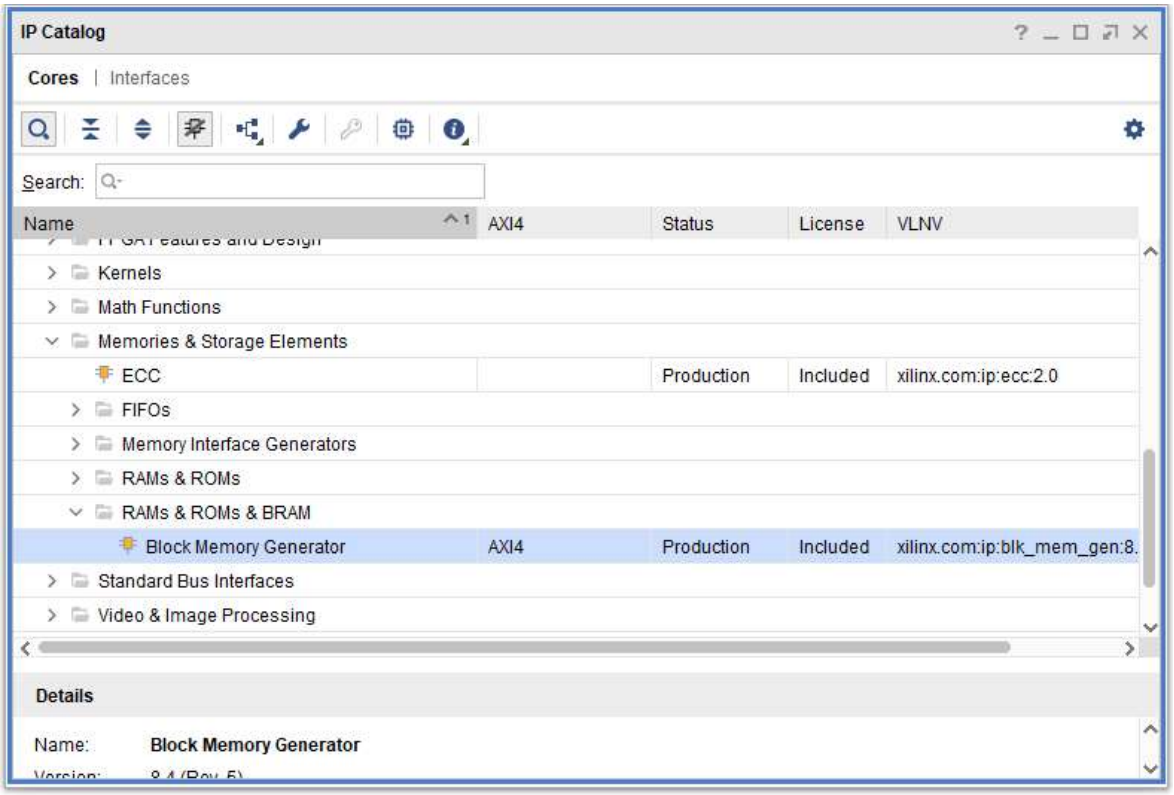
Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger	

Источник - https://github.com/jameslzhu/riscv-card/releases/download/latest/riscv-card.pdf ;

Подробное описание команд можно посмотреть тут - https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view

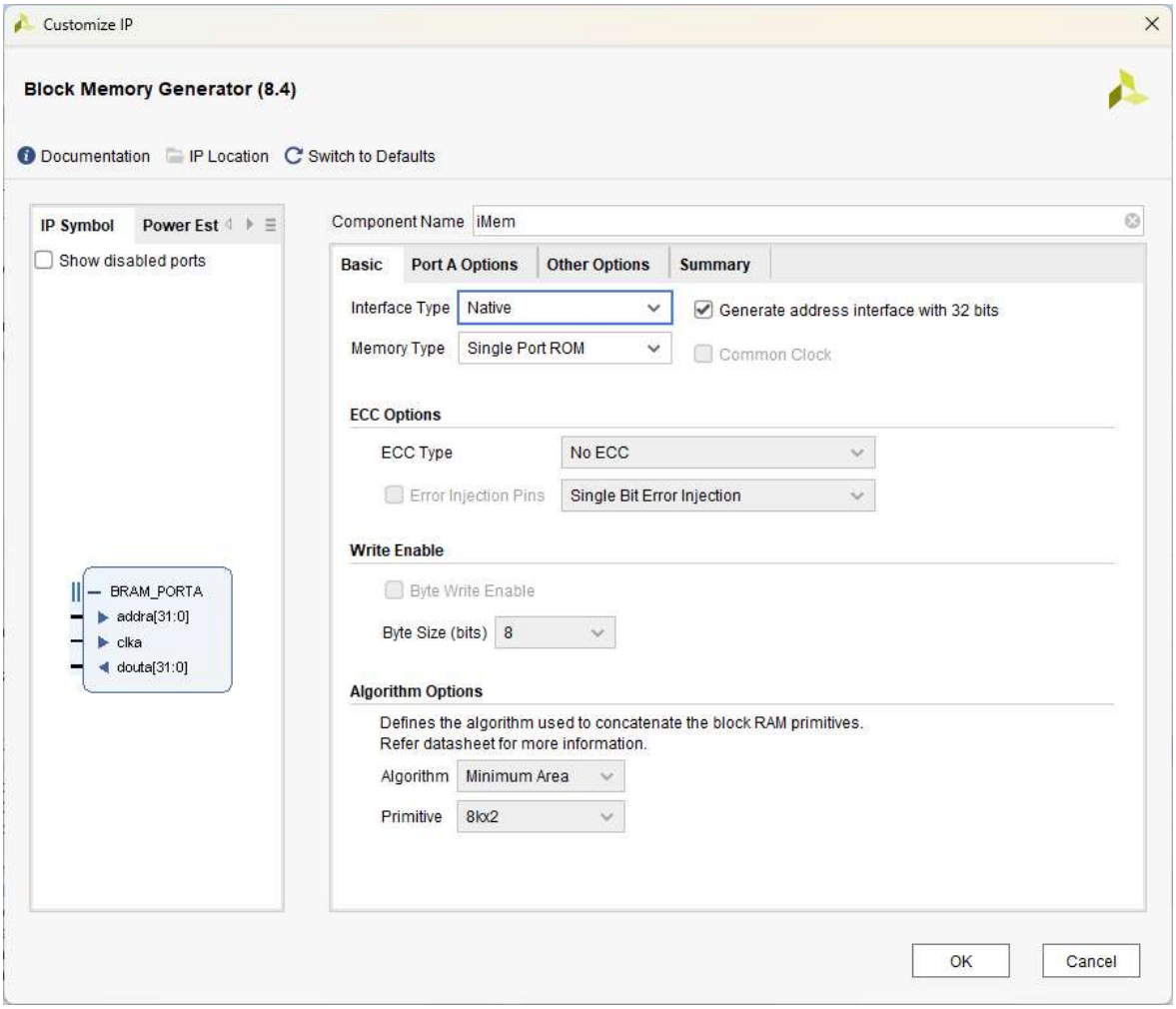
Создание ROM памяти через каталог IP-ядер

Для создания ROM памяти надо открыть каталог IP ядер по аналогии с добавлением VIO. Выбрать в нем пункт **Memories & Storage Elements à RAMs ROMs & BRAM à Block Memory Generator.**

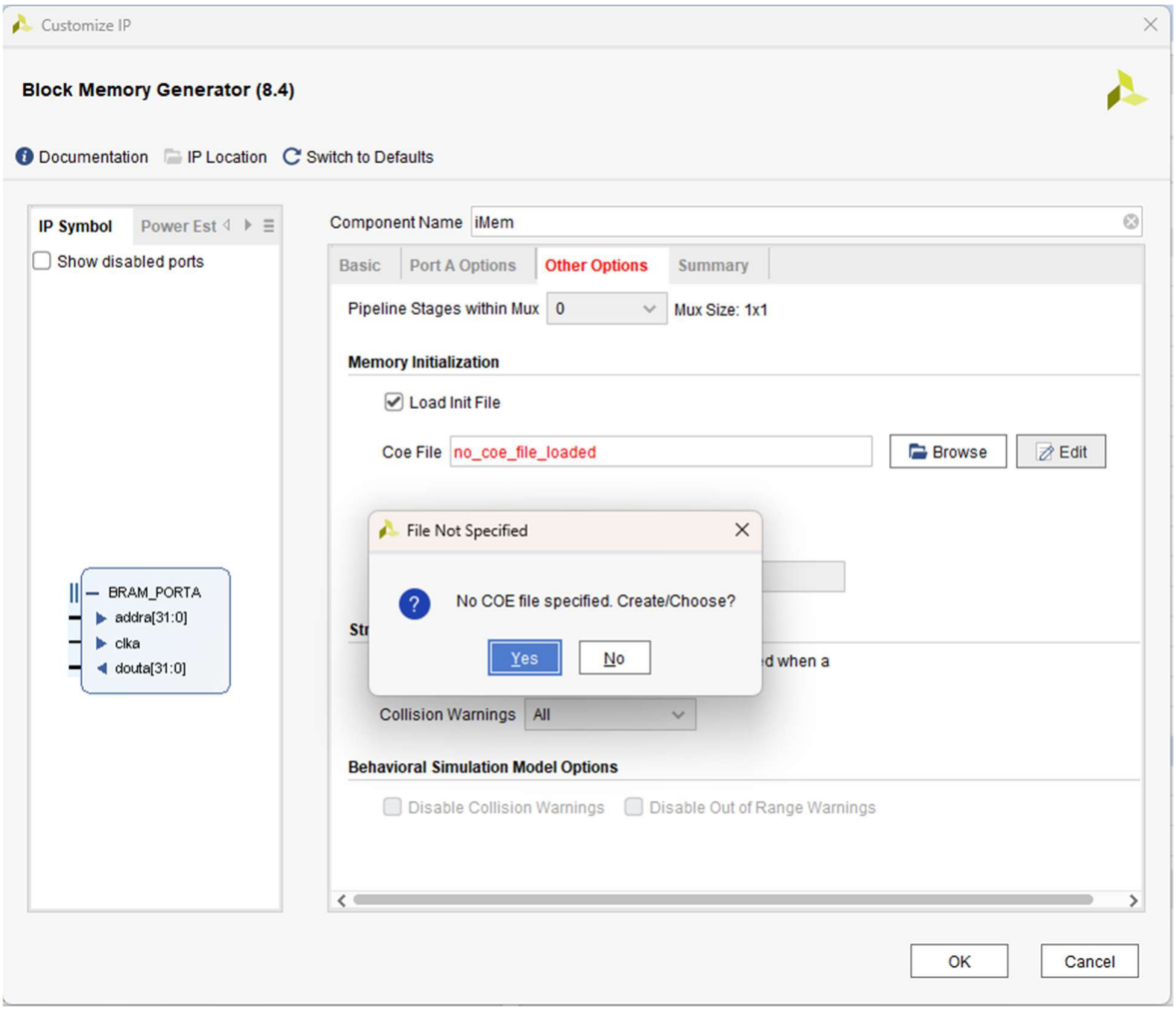


Далее – запустится мастер создания Ip ядра. В первой вкладке необходимо выбрать тип памяти - **Single Port ROM**, и поставить галочку **Generate address interface with 32 bits.** Так же можно задать имя памяти в стройке **Component Name.**

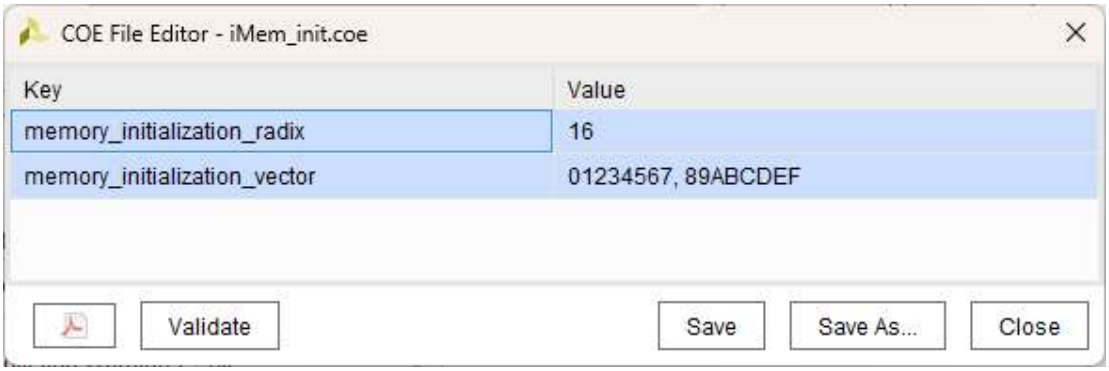
На втором шаге настраивается разрядность памяти, наличие сигнала разрешения чтения и сигналов сброса.



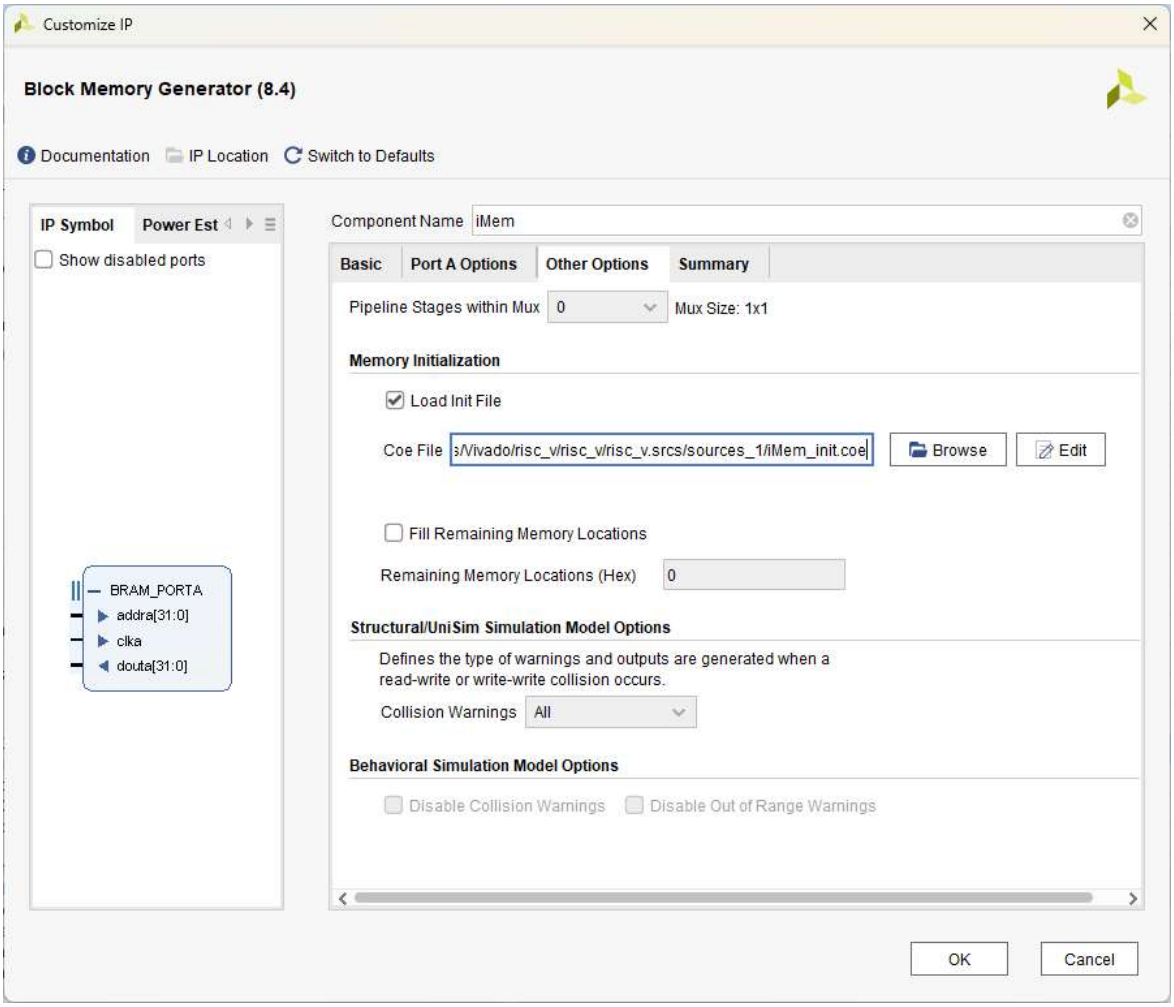
Третий шаг – настройка инициализации памяти. Инициализация выполняется из файла с расширением .coe. Для его создания надо установить галочку **Load init File** и нажать кнопку **Edit.** Мастер предложит создать новый файл для инициализации памяти – надо указать его имя.

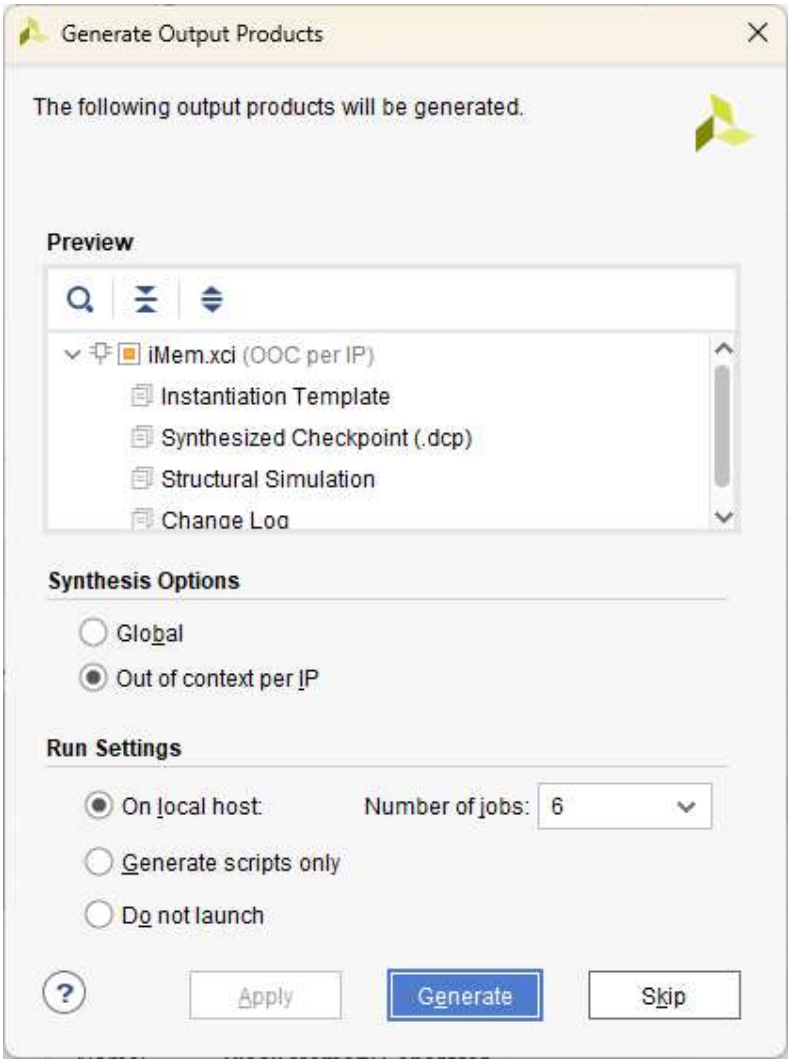


В открывшемся «редакторе» файла. Надо указать систему счисления (2,10,16) и вектор значений через запятую. Можно указать просто пару случайных значений, а потом отредактировать файл в обычном текстовом режиме.

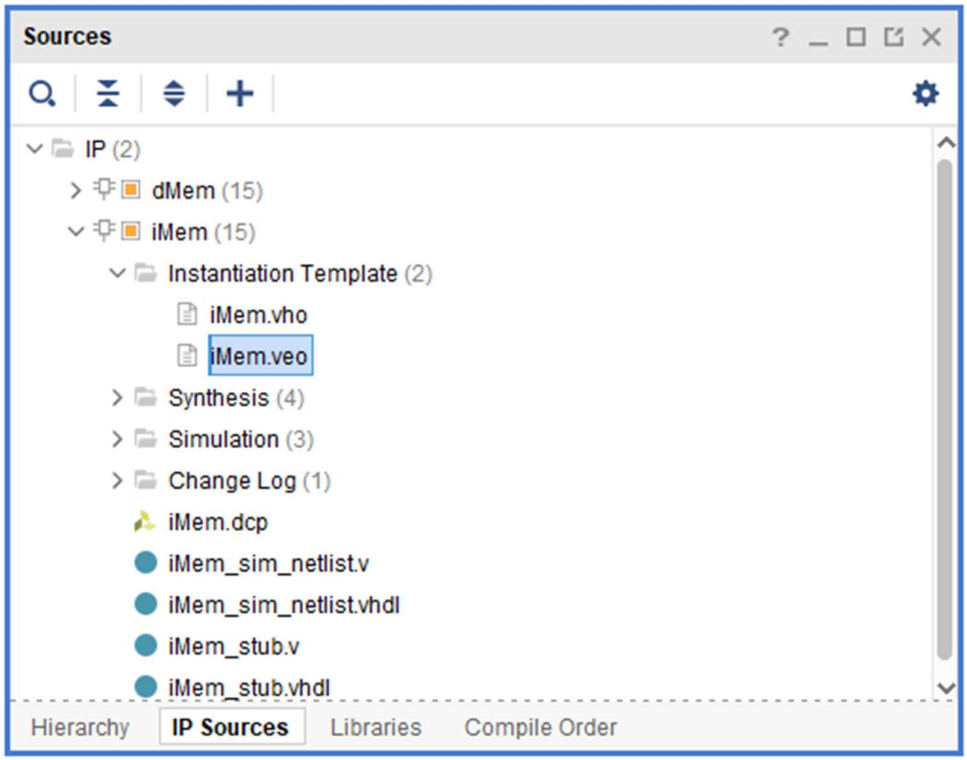


После создания файла его путь появится в соответствующей графе . Последняя вкладка Summary содержит сводную информацию. После нажатия кнопки OK запустится генерация созданного ядра


Customize IP
Block Memory Generator (8.4)
Documentation
IP Location
Switch to Defaults
IP Symbol
Power Est
Show disabled ports
BRAM_PORTA
addra[31:0]
clka
douta[31:0]
Component Name
iMem
Basic
Port A Options
Other Options
Summary
Pipeline Stages within Mux
0
Mux Size: 1x1
Memory Initialization
Load Init File
Coe File
s/Vivado/risc_v/risc_v/risc_v.srcs/sources_1/iMem_init.coe
Browse
Edit
Fill Remaining Memory Locations
Remaining Memory Locations (Hex)
0
Structural/UniSim Simulation Model Options
Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.
Collision Warnings
All
Behavioral Simulation Model Options
Disable Collision Warnings
Disable Out of Range Warnings
OK
Cancel



После завершения генерации ядро будет доступно для подключения в проект, по аналогии с VIO. Шаблон для подключения можно посмотреть во вкладке IP-Sources – имя IP ядра à Instantiation Template à \<NAME>.veo



Файлы начальной инициализации памяти. сое можно редактировать в обычном текстовом режиме.

\<!> После редактирования файлов надо вручную сбросить результаты моделирования (закрыть режим моделирования, Правой кнопкой мыши по **Run Simulation -> Reset Behavioral Simulation** (рис. 18)) и перезапустить его заново – иначе система сама не замечает изменения. сое файлов и будет запускать моделирование со старыми значениями.